CareConnect Outreach

Comprehensive SQL Database Architecture & Management Project

This document details the step-by-step construction, manipulation, and analysis of the CareConnect Outreach database. It demonstrates practical skills in standard SQL (DDL, DML, DQL) used for data engineering and analytics.

Phase 1: Database Architecture & Schema Evolution

Establishing the foundation, structuring the schema, and enforcing data integrity.

Task 1: The Initial Build

Creating the database and the primary table to track medical staff.

```
CREATE DATABASE CareConnect_Outreach;
USE CareConnect_Outreach;

CREATE TABLE Outreach_staff (
    Staff_ID INT,
    Name VARCHAR(100),
    Location VARCHAR(50),
    age INT
);
```

> **Explanation:** This standard DDL (Data Definition Language) query sets up the environment and defines the initial column boundaries and data types (INT for whole numbers, VARCHAR for text strings).

Task 2: Schema Modernization (Multi-Action Alter)

Adapting the table structure dynamically to accommodate new business requirements.

```
ALTER TABLE Outreach_staff
    ADD Monthly_salary INT AFTER Name,
    ADD Department VARCHAR(50),
    MODIFY COLUMN age FLOAT;
```

> **Explanation:** Using a single, optimized ALTER statement reduces server load. The AFTER keyword specifically positions the salary column for better readability, and MODIFY changes the age column to FLOAT to allow for precise decimal ages.

Task 3: Securing Data Integrity

Enforcing rules to prevent bad data entry.

```
ALTER TABLE Outreach_staff
    ADD PRIMARY KEY (Staff_ID);
```

> **Explanation:** Adding a Primary Key constraint ensures that no two staff members can have the same ID and that the ID field can never be left completely empty (NULL).

Phase 2: Data Population & Management (CRUD)

Ingesting real-world data and safely cleaning up testing artifacts.

Task 4: Bulk Data Ingestion

Inserting multiple records into the newly structured table efficiently.

```
INSERT INTO Outreach_staff (Staff_ID, Name, Monthly_salary, Location, age,
Department)
VALUES
(101, 'Nithish', 45000, 'Teynampet', 25.5, 'Pediatrics'),
(102, 'Lavanya', 42000, 'Chennai', 24.0, 'General Medicine'),
(103, 'Ramkumar', 50000, 'Velachery', 29.2, 'Cardiology'),
(104, 'Suresh', 48000, 'Tambaram', 28.0, 'Orthopedics'),
(105, 'Priya', 55000, 'Adyar', 31.0, 'Neurology'),
(106, 'Karthik', 41000, 'Porur', 26.8, 'General Medicine'),
(107, 'Meena', 60000, 'Anna Nagar', 34.5, 'Pediatrics'),
(999, 'Test_user', 10000, 'Madurai', 20.0, 'Admin');
```

> **Explanation:** A multi-row INSERT INTO statement minimizes database transactions. String values are safely wrapped in single quotes.

Task 5: Safe Record Removal

Deleting the temporary test user without impacting live data.

```
DELETE FROM Outreach_staff
WHERE Staff_ID = 999;
```

> **Explanation:** Using the Primary Key (Staff_ID) in the WHERE clause is the safest method to delete a specific row, ensuring no other user named 'Test_user' is accidentally deleted.

Phase 3: Business Logic & Updates

Modifying existing data based on evolving organizational needs.

Task 6: Targeted Bulk Update

Applying a 10% salary hike exclusively to the Pediatrics department.

```
UPDATE Outreach_staff
SET Monthly_salary = Monthly_salary * 1.10
WHERE Department = 'Pediatrics';
```

> **Explanation:** SQL can perform mathematical operations directly. The WHERE clause is critical here to prevent applying the raise to the entire organization.

Task 7: Single Record Multi-Column Update

Processing a promotion and relocation for a specific employee.

```
UPDATE Outreach_staff
SET Location = 'Adyar',
    Monthly_salary = 55000
WHERE Staff_ID = 103;
```

> **Explanation:** Multiple columns can be updated simultaneously by separating SET parameters with commas. Once again, the Primary Key ensures extreme precision.

Phase 4: Analytical Reporting

Extracting specific insights and calculating aggregated metrics.

Task 8: Complex Filtering Logic

Finding staff in specific locations earning above a designated threshold.

```
SELECT * FROM Outreach_staff
WHERE Location IN ('Chennai', 'Adyar')
  AND Monthly_salary > 40000;
```

> **Explanation:** The IN operator is a much cleaner, scalable alternative to writing multiple OR conditions. It is then combined with an AND logic gate to filter the results further.

Task 9: Pattern Matching

Extracting employees whose names start with the letter 'N'.

```
SELECT * FROM Outreach_staff
WHERE Name LIKE 'N%';
```

> **Explanation:** The LIKE operator combined with the wildcard (%) searches for a specific string pattern regardless of how many characters follow it.

Task 10: Data Aggregation

Calculating the average payroll expenditure per employee.

```
SELECT AVG(Monthly_salary) AS Average_Staff_Pay
FROM Outreach_Staff;
```

> **Explanation:** AVG is an aggregate function that crunches a whole column into a single metric. The AS keyword aliases the output column name for a more professional presentation.

Phase 5: Data Security & Backups

Creating dynamic backups before major system operations.

Task 11: Dynamic Table Backup (CTAS)

Generating a filtered backup table for senior staff members.

```
CREATE TABLE Senior_Staff_backup AS
SELECT Name, Location, Monthly_salary
FROM Outreach_staff
WHERE Age > 28;
```

> **Explanation:** The Create Table As Select (CTAS) technique dynamically generates a new table and populates it with queried data in one seamless operation, skipping the need for manual table definition.